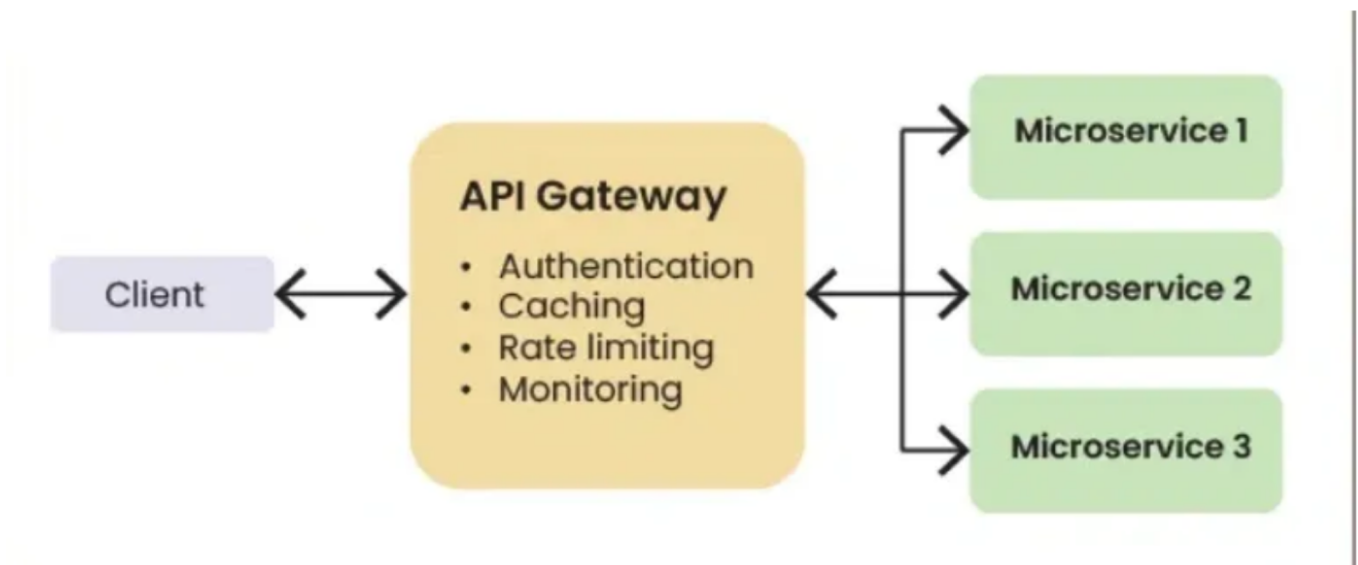


What is API Gateway

DAY 151
08/Dec/25

An API gateway is a key component in System Design, Particularly in Microservices architectures and modern web applications. It serves as a centralized entry point for Managing and routing requests from clients to appropriate Microservices or Backend Services within a System.



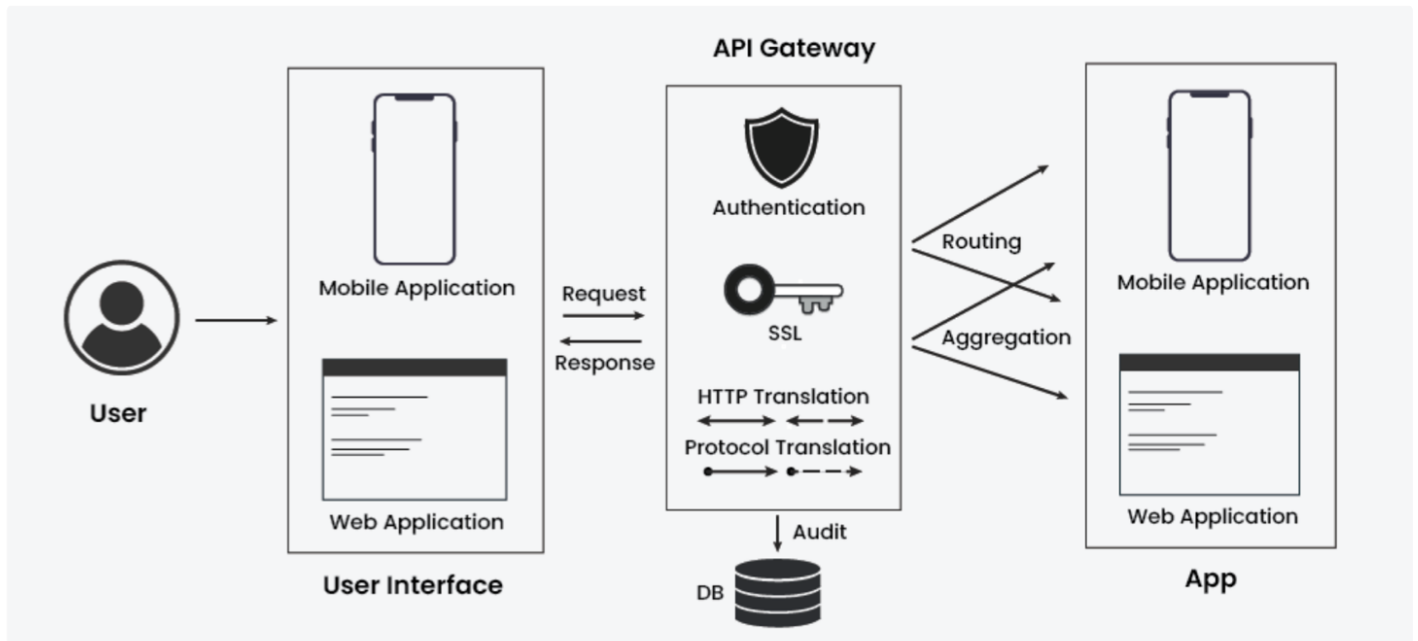
- An API Gateway serves as a reverse proxy between clients and backend services.
- After receiving incoming client requests, it manages a number of responsibilities, including rate limitation, routing, and authentication, before forwarding the requests to the appropriate backend services.
- Offers a consistent interface and hiding the complexity of the underlying architecture

In the above diagram:

- User will send the request from mobile or web application.
- API Gateway will determine which request is coming.
- Authentication means the user need to proof there identity to the server or client, by providing there User_Id and Password. For example: Login or Signup page.
- SSL full form Secure Socket Layer, it is used to establish an encrypted link between a server and a client.
- It provides the ability to perform protocol translation, where incoming requests are translated from one channel to another.
- When requests are aggregated, a request received by an API gateway will trigger requests to different endpoints, and return response to the client.

The primary purpose of an API Gateway is to simplify the client's interaction with the underlying services, enhance

security, and provide various features for managing and monitoring API traffic.



How does API Gateway work?

Let us see how API Gateway works:

- **Routing** – Directs client requests to the appropriate service based on URL, method, or headers.
- **Protocol Translation** – Converts requests between protocols (e.g., HTTP → gRPC/WebSocket).
- **Request Aggregation** – Combines multiple backend calls into one to reduce round trips.
- **Authentication & Authorization** – Verifies client identity and access permissions.
- **Rate Limiting & Throttling** – Controls request rates to prevent abuse and ensure resource balance.
- **Load Balancing** – Distributes requests across service instances for scalability and availability.
- **Caching** – Stores backend responses to speed up repeated requests.
- **Monitoring & Logging** – Tracks metrics and logs for performance and usage insights.